

DS701 Homework 3: k-means and Linkage

Released Mon Sep 21 · due Mon Sep 28, 11:59 pm on Gradescope

SOLUTIONS

Problem 1: Lloyd's algorithm by hand

Practices: running k-means step by step, tracking the objective, and seeing a bad local minimum.

Six one-dimensional data points:

$$x = 0, 5, 6, 7, 12, 15.$$

Cluster them with k -means, $K = 2$, using Euclidean distance (on a line, $\|x - c\|^2 = (x - c)^2$). The objective is the WCSS / inertia, $\sum_k \sum_{x \in C_k} (x - c_k)^2$.

- (a) Start from the centers $c_1 = 12$, $c_2 = 15$. Run **iteration 1**: assign every point to its nearest center, then update each center to the mean of its cluster. Report the two clusters, the WCSS *right after the assignment step* (using the old centers), the new centers, and the WCSS *right after the update step* (using the new centers, same assignment).

Iteration 1	Cluster 1	Cluster 2
Points assigned	_____	_____
WCSS after assignment (old centers)	_____	_____
New centers	_____	_____
WCSS after update (new centers)	_____	_____

- (b) Run **iteration 2** the same way and report the same four things. Then state whether a third assignment step would change anything, and list the WCSS values you have seen, in order. Did the objective ever go up?

Iteration 2	Cluster 1	Cluster 2
Points assigned	_____	_____
WCSS after assignment (old centers)	_____	_____
New centers	_____	_____
WCSS after update (new centers)	_____	_____

Would a third assignment step change anything? _____

WCSS values seen, in order: _____ Ever went up? _____

- (c) Now start instead from $c_1 = 0, c_2 = 5$. Run Lloyd's algorithm until the centers stop changing (it is short). Report the final clusters, centers, and WCSS. Compare with (b): which run found the better clustering, and is the other run "wrong" from the algorithm's point of view?

Final clusters: _____ Centers: _____ WCSS: _____

- (d) Two sentences: what does (c) imply for how k -means is run in practice, and what would the k -means++ idea from the lecture have done with this data? (Check your claim: run one iteration from the two centers k -means++ would most plausibly pick.)

Solution

- (a) Distances to $c_1 = 12$ vs. $c_2 = 15$: 0, 5, 6, 7 and 12 are nearer to 12; only 15 is nearer to 15. Clusters $\{0, 5, 6, 7, 12\}, \{15\}$. WCSS after assignment (old centers): $144 + 49 + 36 + 25 + 0 + 0 = 254$. Update: $c_1 = (0 + 5 + 6 + 7 + 12)/5 = 6, c_2 = 15$. WCSS after update: $36 + 1 + 0 + 1 + 36 + 0 = 74$.
- (b) Distances to 6 vs. 15: 0, 5, 6, 7 $\rightarrow c_1$ (7: 1 vs. 8); 12 $\rightarrow c_2$ (6 vs. 3); 15 $\rightarrow c_2$. Clusters $\{0, 5, 6, 7\}, \{12, 15\}$. WCSS after assignment (centers 6, 15): $36 + 1 + 0 + 1 + 9 + 0 = 47$. Update: $c_1 = 18/4 = 4.5, c_2 = 27/2 = 13.5$. WCSS after update: $20.25 + 0.25 + 2.25 + 6.25 + 2.25 + 2.25 = 33.5$. A third assignment step changes nothing (7: 2.5 vs. 6.5; 12: 7.5 vs. 1.5), so the centers stay at 4.5, 13.5: converged. Sequence: $254 \rightarrow 74 \rightarrow 47 \rightarrow 33.5$ — never increased. (Common mistake: computing the WCSS only after the update and so never seeing that *both* steps lower it.)
- (c) From $c_1 = 0, c_2 = 5$: $0 \rightarrow c_1$; 5, 6, 7, 12, 15 $\rightarrow c_2$. WCSS with old centers $0 + 0 + 1 + 4 + 49 + 100 = 154$; new centers $c_1 = 0, c_2 = 45/5 = 9$; WCSS = $0 + 16 + 9 + 4 + 9 + 36 = 74$. Next assignment: 5 is at distance 5 from 0 and 4 from 9, so nothing moves; converged at clusters $\{0\}, \{5, 6, 7, 12, 15\}$, centers 0, 9, WCSS = 74. The run in (b) is better ($33.5 < 74$; a brute-force check over all splits confirms 33.5 is the global minimum). The run in (c) is not "wrong" for the algorithm: it satisfies both stopping conditions (every point is at its nearest center and every center is its cluster's mean) — it is a local minimum of the objective. Lloyd's algorithm has no way to tell.
- (d) Because the result depends on the start, run k -means from several random initializations and keep the run with the lowest WCSS (`n_init` in `sklearn`); or choose starting centers that are far apart (k -means++), which is what `sklearn` does by default. Here k -means++ would most plausibly pick 0 and 15; from (0, 15) one iteration gives clusters $\{0, 5, 6, 7\}, \{12, 15\}$ (7: 7 vs. 8), centers 4.5, 13.5, WCSS 33.5 — the optimum, in one step.

Problem 2: Why k -means stops, and why more clusters always "help"

Practices: the two one-line facts behind Lloyd's algorithm, and reading the WCSS correctly when choosing K .

Write $J(C, c) = \sum_{k=1}^K \sum_{x \in C_k} \|x - c_k\|^2$ for the objective, where $C = (C_1, \dots, C_K)$ is the assignment of points to clusters and $c = (c_1, \dots, c_K)$ the centers.

- (a) **Assignment step.** Hold the centers c fixed and let C' be the assignment that sends every point to its nearest center. Show that $J(C', c) \leq J(C, c)$ for *any* assignment C . (One or two lines: compare the two sums term by term.)

- (b) **Update step.** Hold an assignment fixed and look at one cluster with points $x_1, \dots, x_n$ (one-dimensional, to keep the algebra short — the same argument works coordinate by coordinate). Show that $f(c) = \sum_{i=1}^n (x_i - c)^2$ is minimized at $c = \bar{x} = \frac{1}{n} \sum_i x_i$. Either differentiate f and set the derivative to zero (and say why that critical point is a minimum), or expand $(x_i - c)^2 = ((x_i - \bar{x}) + (\bar{x} - c))^2$ and simplify.
- (c) **Termination.** Using (a), (b), and one further fact — that there are only finitely many ways to assign N points to K clusters — explain in a short paragraph why Lloyd’s algorithm must stop after finitely many iterations (assume, as `sklearn` does, that a point stays where it is when it is equally close to two centers). Then say in one sentence what the argument does *not* promise about the clustering it stops at.
- (d) **Choosing K .** Let J_K^* be the smallest WCSS achievable with K clusters (the global optimum, not a particular run). Explain why $J_{K+1}^* \leq J_K^*$ for every K , and what J_N^* is when N is the number of points. Conclude why “pick the K with the smallest WCSS” is not a usable rule, and name the criterion the lecture uses instead — describe in one sentence what it looks for.

$J_N^* = \underline{\hspace{2cm}}$ Criterion the lecture uses: $\underline{\hspace{2cm}}$

Solution

- (a) $J(C, c) = \sum_x \|x - c_{a(x)}\|^2$, where $a(x)$ is the cluster x is assigned to under C . For each point, $\|x - c_{a'(x)}\|^2 = \min_k \|x - c_k\|^2 \leq \|x - c_{a(x)}\|^2$, because $a'(x)$ is *defined* to be the minimizing index. Summing N inequalities that each go the right way gives $J(C', c) \leq J(C, c)$.
- (b) *Calculus:* $f'(c) = -2 \sum_i (x_i - c) = -2(\sum_i x_i - nc) = 0 \Rightarrow c = \frac{1}{n} \sum_i x_i = \bar{x}$; and $f''(c) = 2n > 0$ (or: f is a sum of upward parabolas in c), so this critical point is the global minimum. *Identity:*

$$\sum_i (x_i - c)^2 = \sum_i (x_i - \bar{x})^2 + 2(\bar{x} - c) \sum_i (x_i - \bar{x}) + n(\bar{x} - c)^2 = \sum_i (x_i - \bar{x})^2 + n(\bar{x} - c)^2,$$

since $\sum_i (x_i - \bar{x}) = 0$. The first term does not depend on c and the second is ≥ 0 with equality iff $c = \bar{x}$. In d dimensions the sum splits over coordinates, so the minimizer is the coordinate-wise mean, i.e. the centroid.

- (c) By (a) the assignment step never increases J and by (b) the update step never increases J , so J is non-increasing along the run. Moreover, if the assignment step actually moves some point (with the stay-put tie rule, it moves only when it becomes strictly closer to another center), J strictly decreases, so an assignment, once left, can never recur — the algorithm never revisits an assignment. There are at most K^N assignments, so the run can change assignments at most K^N times and must stop. (Practically it stops after tens of iterations, not K^N ; the bound only proves *finiteness*.) What it does not promise: that the final clustering is the global minimum of J — Problem 1(c) is a fixed point with WCSS 74 while the optimum is 33.5.
- (d) Take an optimal K -clustering and add any data point that is not already a center as a $(K+1)$ -st center. Every point’s distance to its nearest center can only shrink or stay the same, so $J \leq J_K^*$; the optimum over $K+1$ centers is at most this, hence $J_{K+1}^* \leq J_K^*$. With $K = N$ every point is its own center and $J_N^* = 0$. So the WCSS decreases all the way to zero and “smallest WCSS” would always answer $K = N$, which is useless. The lecture instead uses the **elbow**: plot J_K^* (in practice, the best-of-restarts WCSS) against

K and pick the K after which the curve stops dropping steeply — the point of diminishing returns. (The lecture 04 material also scores flat clusterings by silhouette; either answer is fine if described correctly.)

Problem 3: Single vs. complete linkage on five points

Practices: agglomerative clustering step by step, dendrogram heights, chaining, and matching a linkage to a data shape.

Five objects A–E with pairwise distances (a metric):

	A	B	C	D	E
A	0	2	5	9	12
B	2	0	3	7	10
C	5	3	0	4	8
D	9	7	4	0	5
E	12	10	8	5	0

- (a) Perform agglomerative clustering with **single linkage**. At each of the four steps write the current cluster-to-cluster distances that matter, the pair merged, and the merge height. Then write the result as the 4×4 linkage matrix, rows `[idx1, idx2, dist, count]`, with A–E as indices 0–4 and new clusters numbered 5, 6, ... in creation order.

Single linkage	Pair merged	Height	Linkage-matrix row
Step 1	_____	_____	_____
Step 2	_____	_____	_____
Step 3	_____	_____	_____
Step 4	_____	_____	_____

- (b) Repeat with **complete linkage**.

Complete linkage	Pair merged	Height	Linkage-matrix row
Step 1	_____	_____	_____
Step 2	_____	_____	_____
Step 3	_____	_____	_____
Step 4	_____	_____	_____

- (c) Sketch both dendrograms (leaves on the x-axis, merge heights on the y-axis). For each, say which two clusters you get by cutting the tree into two flat clusters. Then explain, in terms of the *definitions* of the two linkages, why single linkage swallowed A, B, C, D one at a time while complete linkage kept two compact groups — the lecture’s word for the single-linkage behavior is *chaining*.

Two-cluster cut, single: _____ complete: _____

- (d) For each situation below, name the linkage the lecture would recommend and give one sentence of justification: (i) two crescent-shaped (“half-moon”) clusters that curve around each other; (ii) several roughly round groups of similar size, plus a handful of scattered noise points between them.

Situation (i) — linkage: _____

Situation (ii) — linkage: _____

Solution

- (a) *Single linkage* (cluster distance = smallest cross-pair distance). Step 1: the smallest entry is $d(A, B) = 2 \rightarrow$ merge A, B, height 2. Step 2: $d(\{AB\}, C) = \min(5, 3) = 3$, $d(\{AB\}, D) = \min(9, 7) = 7$, $d(\{AB\}, E) = 10$, $d(C, D) = 4$, $d(C, E) = 8$, $d(D, E) = 5 \rightarrow$ merge $\{AB\}$, C, height 3. Step 3: $d(\{ABC\}, D) = \min(9, 7, 4) = 4$, $d(\{ABC\}, E) = 8$, $d(D, E) = 5 \rightarrow$ merge $\{ABC\}$, D, height 4. Step 4: $d(\{ABCD\}, E) = \min(12, 10, 8, 5) = 5 \rightarrow$ merge, height 5. Linkage matrix: $[0, 1, 2, 2], [2, 5, 3, 3], [3, 6, 4, 4], [4, 7, 5, 5]$.
- (b) *Complete linkage* (cluster distance = largest cross-pair distance). Step 1: merge A, B at 2 (same first step — singletons have only one cross pair). Step 2: $d(\{AB\}, C) = \max(5, 3) = 5$, $d(\{AB\}, D) = 9$, $d(\{AB\}, E) = 12$, $d(C, D) = 4$, $d(C, E) = 8$, $d(D, E) = 5 \rightarrow$ merge C, D, height 4. Step 3: $d(\{AB\}, \{CD\}) = \max(5, 9, 3, 7) = 9$, $d(\{AB\}, E) = 12$, $d(\{CD\}, E) = \max(8, 5) = 8 \rightarrow$ merge $\{CD\}$, E, height 8. Step 4: $d(\{AB\}, \{CDE\}) = \max(5, 9, 12, 3, 7, 10) = 12 \rightarrow$ merge, height 12. Linkage matrix: $[0, 1, 2, 2], [2, 3, 4, 2], [4, 6, 8, 3], [5, 7, 12, 5]$. (Common mistake: after merging, forgetting to recompute the cluster distance with the max, and reusing the min-based value 3 for $\{AB\}$ –C.)
- (c) Single: a staircase — A and B join at 2, then C at 3, D at 4, E at 5; each step adds one leaf to the same growing cluster. Two-cluster cut (between heights 4 and 5): $\{A, B, C, D\}$ and $\{E\}$. Complete: two low joins, A–B at 2 and C–D at 4; E joins $\{CD\}$ at 8; the two branches join at 12. Two-cluster cut (between 8 and 12): $\{A, B\}$ and $\{C, D, E\}$. Why: single linkage merges as soon as *one* cross pair is close, so a run of points each near the next (A–B 2, B–C 3, C–D 4, D–E 5) is absorbed link by link even though the ends are far apart ($d(A, D) = 9$) — chaining. Complete linkage charges a merge the *farthest* cross pair, i.e. the diameter of the merged group, so pulling C into $\{AB\}$ costs 5 while pairing C with D costs 4; it keeps groups compact and only merges the two compact groups at the end, at 12.
- (d) (i) **Single** linkage: a crescent is a chain of nearby points, and single linkage follows the nearest-neighbor chain along the arc, so it recovers non-elliptical shapes (the lecture’s half-moon example); complete/average would cut the crescents into round pieces. (ii) **Complete** or **average** (or Ward): noise points between round groups would form a bridge that single linkage chains across, merging the groups; complete/average linkage judge a merge by the far (or average) cross distance, so a few bridge points cannot pull two clusters together, and they produce compact, similar-diameter clusters. Average/Ward is the noise-robust compromise; complete is acceptable if the noise points are not extreme outliers (it is outlier-sensitive).